

Branch • Python Script to Deduplicate Video Frames

<https://gemini.google.com/app/40bbb7bb2eee386b>

User prompt: Give me a python script, where given a hardcoded file, split the file into images under a subfolder of the video named 'images', then delete duplicate frames. Make sure the frames stay in chronological order.

Response: To split a video into frames and automatically purge duplicate images while preserving the original chronological order, you can use **OpenCV** to extract frames and **imagehash** to perform perceptual hashing (dhash) for duplicate detection.

Prerequisites

Install the required libraries first:

```
pip install opencv-python imagehash pillow
```

Python Script

Save the script in the same directory as your video file and run it:

```
import os
import cv2
from PIL import Image
import imagehash

# Configuration
VIDEO_PATH = "sample_video.mp4" # Path to your video file
HASH_THRESHOLD = 4 # Lower = stricter matching, higher = looser matching

def extract_and_deduplicate(video_path, hash_threshold=4):
    if not os.path.exists(video_path):
        print(f"Error: Video file '{video_path}' not found.")
        return

    # Set up output directory under 'images/' relative to video directory
    video_dir = os.path.dirname(os.path.abspath(video_path))
    output_dir = os.path.join(video_dir, "images")
    os.makedirs(output_dir, exist_ok=True)

    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        print("Error: Could not open video.")
        return

    frame_count = 0
    saved_count = 0
    last_saved_hash = None

    print("Processing video frames...")

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        frame_count += 1

        # Convert OpenCV BGR frame to PIL Image for imagehash
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        pil_img = Image.fromarray(rgb_frame)

        # Compute difference hash (dhash)
        current_hash = imagehash.dhash(pil_img)

        # Check if current frame is a duplicate of the last saved frame
        if last_saved_hash is not None:
            hash_diff = current_hash - last_saved_hash
            if hash_diff <= hash_threshold:
                # Skip duplicate frame
                continue

        # Save frame sequentially to maintain chronological order
```

```

    saved_count += 1
    output_filename = os.path.join(output_dir, f"frame_{saved_count:05d}.jpg")
    cv2.imwrite(output_filename, frame)

    # Update the hash of the last unique frame saved
    last_saved_hash = current_hash

cap.release()
print(f"Done! Processed {frame_count} total frames.")
print(f"Saved {saved_count} unique frames to '{output_dir}'.")

if __name__ == "__main__":
    extract_and_deduplicate(VIDEO_PATH, HASH_THRESHOLD)

```

How It Works

- **Chronological Order:** Frames are extracted sequentially in time and named `frame_00001.jpg`, `frame_00002.jpg`, etc., using zero-padded numbers to preserve alphabetical and time ordering.
- **Perceptual Hashing (dhash):** Instead of comparing exact pixel matches (which fail due to minor video compression noise), perceptual hashing calculates a visual footprint of each frame.
- **HASH_THRESHOLD Tuning:**
 - 0: Exact match only (may miss near-identical duplicates with compression artifacts).
 - 1–5: Recommended threshold range for detecting near-identical or identical frames.
 - 6+: Drops frames with subtle movement or panning.

User prompt: I would like to modify the above a bit. I am now working off a structure of m4s files with a session.mpd file. I would like to only compare the bottom left of the image. I would like to do this by first cropping the image to crop top left (1582, 873) crop top right (1755, 873) crop bottom left (1582, 1083) crop bottom right (1755, 1038) the crop is assumed from the top left being 0,0. I would like the images to be in order and if the name of the clip that produces the image could be added that would be apricated, at bare minimum a timestamp added to the bottom of the image crop so I can identify the location of the cropped image in the played sequence.

Response: Here is the updated Python script based on your original `clipping.py`. It is designed to work directly with the `session.mpd` file that manages the `.m4s` chunks shown in your `tree.txt` folder structure.

Key Adjustments Made:

- **Cropping:** OpenCV crops using rectangular numpy array slicing (`frame[y1:y2, x1:x2]`). Based on your coordinates, I set the bounding box to X: 1582 to 1755 and Y: 873 to 1083 (using the maximum bottom-left y value to ensure the whole area is captured, as 1083 and 1038 differed).
- **Targeted Hashing:** The `imagehash.dhash` function now solely evaluates the *cropped* region, meaning changes in the broader video will be ignored unless the bottom-left corner changes.
- **Timestamp Overlay:** OpenCV retrieves the current playback position in milliseconds. The script formats this into an `HH:MM:SS.mmm` timestamp and stamps it directly onto the bottom of the cropped image before saving.

The Modified Script

```

import os
import cv2
import datetime
from PIL import Image
import imagehash
from pathlib import Path

# Configuration
HASH_THRESHOLD = 4 # Lower = stricter matching, higher = looser matching

# Crop Coordinates (Top Left: 1582, 873 | Bottom Right: 1755, 1083)
# Note: Using 1083 instead of 1038 to ensure the bounding box covers the largest Y axis requested
CROP_Y1 = 873
CROP_Y2 = 1083
CROP_X1 = 1582
CROP_X2 = 1755

```

```

def format_timestamp(ms):
    """Converts milliseconds into a readable HH:MM:SS.mmm string."""
    if ms < 0:
        return "Unknown Time"
    td = datetime.timedelta(milliseconds=ms)
    # Format to keep it concise and include milliseconds
    return str(td)[-3]

def extract_and_deduplicate(output_dir, video_path, hash_threshold=4):
    if not os.path.exists(video_path):
        print(f"Error: Video file '{video_path}' not found.")
        return

    os.makedirs(output_dir, exist_ok=True)

    # OpenCV will use FFmpeg backend to read the session.mpd and parse the .m4s chunks
    cap = cv2.VideoCapture(str(video_path))
    if not cap.isOpened():
        print("Error: Could not open video/mpd stream.")
        return

    frame_count = 0
    saved_count = 0
    last_saved_hash = None

    print("Processing video frames and cropping...")

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        frame_count += 1

        # 1. Crop the frame to the specified bottom-left coordinates
        # OpenCV frames are numpy arrays indexed via [y1:y2, x1:x2]
        cropped_frame = frame[CROP_Y1:CROP_Y2, CROP_X1:CROP_X2]

        # 2. Convert cropped OpenCV BGR frame to PIL Image for imagehash
        rgb_crop = cv2.cvtColor(cropped_frame, cv2.COLOR_BGR2RGB)
        pil_img = Image.fromarray(rgb_crop)

        # 3. Compute difference hash (dhash) on the CROP only
        current_hash = imagehash.dhash(pil_img)

        # 4. Check if current cropped frame is a duplicate
        if last_saved_hash is not None:
            hash_diff = current_hash - last_saved_hash
            if hash_diff <= hash_threshold:
                # Skip duplicate frame
                continue

        # 5. Get the timestamp from the video stream
        timestamp_ms = cap.get(cv2.CAP_PROP_POS_MSEC)
        time_str = format_timestamp(timestamp_ms)

        # 6. Overlay the timestamp onto the cropped image
        # Positioned at the bottom left of the crop
        text_position = (5, cropped_frame.shape[0] - 10)
        cv2.putText(
            cropped_frame,
            time_str,
            text_position,
            cv2.FONT_HERSHEY_SIMPLEX,
            0.5,          # Font scale
            (0, 255, 0),   # Color (Green)
            1,            # Thickness
            cv2.LINE_AA
        )

        # 7. Save frame sequentially to maintain chronological order
        saved_count += 1

```

```

output_filename = os.path.join(output_dir, f"frame_{saved_count:05d}.jpg")
cv2.imwrite(output_filename, cropped_frame)

# Update the hash of the last unique frame saved
last_saved_hash = current_hash

cap.release()
print(f"Done! Processed {frame_count} total frames.")
print(f"Saved {saved_count} unique cropped frames to '{output_dir}'.")

if __name__ == "__main__":
    video = Path(
        r"C:\Program Files (x86)\Steam\userdata\279248004\gamerecordings\video\BG_1938090_20260917_033631\session.mpd"
    )
    ws = Path(__file__).parent

    extract_and_deduplicate(
        output_dir=ws / "images",
        video_path=video,
        hash_threshold=HASH_THRESHOLD,
    )

```

Depending on the specific FFmpeg backend compiled with your system's OpenCV installation, reading a `session.mpd` directly might sometimes result in an inaccurate `CAP_PROP_POS_MSEC` return (returning `0.0` or `-1`). If you notice the printed timestamps on the images are stuck at `0:00:00.000`, the easiest fallback is to calculate the time manually by dividing the `frame_count` variable by the video's FPS (`cap.get(cv2.CAP_PROP_FPS)`).

User prompt: `` (find-sniper) PS C:\Users\GrayJ\dev\find\_sniper> & c:/Users/GrayJ/dev/find\_sniper/.venv/Scripts/python.exe -B c:/Users/GrayJ/dev/find\_sniper/clipping.py Error: Could not open video/mpd stream. (find-sniper) PS C:\Users\GrayJ\dev\find\_sniper> `` what information do you need to help debug this issue?

Response: Because `os.path.exists()` passed in your script, we know the path is correct and the file exists. The failure happens at `cv2.VideoCapture()`.

The most common reason for this specific error is a **working directory mismatch**. DASH manifests (`.mpd`) typically use relative paths to point to their `.m4s` chunks. Because you are executing the script from `C:\Users\GrayJ\dev\find_sniper`, OpenCV's FFmpeg backend is likely looking for the `.m4s` chunks in your script's folder rather than the Steam folder.

1. The Immediate Fix to Try

Before diving into deep debugging, try changing the script's current working directory to the folder containing the video files right before opening the capture stream.

Add this single line to `extract_and_deduplicate` in `clipping_2.py`:

```

os.makedirs(output_dir, exist_ok=True)

# ADD THIS LINE: Change working directory to where the mpd and m4s files live
os.chdir(os.path.dirname(video_path))

# OpenCV will use FFmpeg backend to read the session.mpd and parse the .m4s chunks
cap = cv2.VideoCapture(str(video_path))

```

To verify if this worked, simply run the script again and see if it begins printing "Processing video frames and cropping..." instead of the stream error.

2. Information Needed if the Fix Fails

If the above change still results in `Error: Could not open video/mpd stream.`, the standard `opencv-python` package you installed may not have been compiled with the necessary FFmpeg DASH demuxer enabled.

To debug further, please provide the following:

1. **The contents of `session.mpd`:** Open the file in a text editor and provide the first 20-30 lines so I can verify how it references the `.m4s` chunks.
2. **Your OpenCV Build Info:** Run the following command in your terminal and share the output under the `Video I/O:` section so I can see your FFmpeg configuration:

```
c:/Users/GrayJ/dev/find_sniper/.venv/Scripts/python.exe -c "import cv2; print(cv2.getBuildInformation())"
```

3. **Standalone FFmpeg Status:** Do you currently have the standalone `ffmpeg` CLI tool installed on your system and accessible in your system PATH? (If OpenCV cannot read the MPD, we can use standard FFmpeg to merge the chunks into an `.mp4` transparently).

User prompt: I have stand alone ffmpeg and vlc, how can I convert the session to a mkv or mp4 to make it easier for the script to process?

Response: Using standalone FFmpeg is the cleanest and fastest approach here. Since you already have the chunks and the manifest (`session.mpd`), you can use FFmpeg to multiplex them into a single `.mp4` file without re-encoding the video. This preserves the original quality and usually takes only a few seconds.

Here is how to do it using your PowerShell terminal.

Merge the stream with FFmpeg

Because `session.mpd` relies on relative paths to find the `.m4s` chunks, you must change your working directory to the folder containing the video files before running the FFmpeg command.

Run these commands in your PowerShell terminal:

```
# 1. Navigate to the folder containing the session.mpd and .m4s files
cd "C:\Program Files (x86)\Steam\userdata\279248004\gamerecordings\video\bg_1938090_20260917_000457"

# 2. Tell FFmpeg to read the manifest and copy the streams into an MP4 container
ffmpeg -i session.mpd -c copy output.mp4
```

How to verify this worked: Type `ls` in that directory and you should see a newly created `output.mp4` file. You can double-click this file to open it in VLC or the default Windows media player to ensure it plays correctly and the duration matches what you expect.

Update your Python Script

Once the file is generated, you just need to update the file path at the bottom of your `clipping_2.py` script to point to the new `.mp4` instead of the `.mpd`.

```
if __name__ == "__main__":
    video = Path(
        r"C:\Program Files (x86)\Steam\userdata\279248004\gamerecordings\video\bg_1938090_20260917_000457\output.mp4"
    )
    ws = Path(__file__).parent

    extract_and_deduplicate(
        output_dir=ws / "images",
        video_path=video,
        hash_threshold=HASH_THRESHOLD,
    )
```

User prompt: Can this process be parallized? I have many cores and would like to use them all if possible. I also have an AMD gpu if that helps.

Response: Yes, we can absolutely parallelize this to utilize all your CPU cores.

Because we must keep the frames in chronological order and our deduplication relies on knowing the hash of the *previous* frame, we cannot process the saving logic randomly. However, computing the perceptual hash (`imagehash.dhash`) is by far the most CPU-intensive part of the script.

We can use a **Producer-Consumer Batching** approach using `concurrent.futures.ProcessPoolExecutor`:

1. The **Main Thread** rapidly reads the video, crops the frames, and groups them into batches.
2. The **Worker Processes** (using all your CPU cores) simultaneously compute the complex hashes for the batch.
3. The **Main Thread** receives the processed hashes back *in their exact original order*, allowing it to safely perform the stateful deduplication and save the images.

Regarding your AMD GPU

While OpenCV can sometimes utilize hardware acceleration for video decoding, standard pre-compiled Python binaries prioritize Nvidia (CUDA). Furthermore, the `imagehash` and `PIL` libraries run exclusively on the CPU. By moving the heavy hashing logic to a multiprocessing pool, you will easily max out your CPU cores and dramatically reduce processing time, making GPU offloading unnecessary for this specific workload.

The Multi-Core Script

Replace your Python script with this updated version. Note that the `compute_hash` function has been moved to the top level—this is a strict requirement for Windows multiprocessing.

```

import os
import cv2
import datetime
import multiprocessing
from concurrent.futures import ProcessPoolExecutor
from PIL import Image
import imagehash
from pathlib import Path

# Configuration
HASH_THRESHOLD = 4
BATCH_SIZE = 300 # Number of frames to hold in memory per multiprocessing batch

# Crop Coordinates (Top Left: 1582, 873 | Bottom Right: 1755, 1083)
CROP_Y1 = 873
CROP_Y2 = 1083
CROP_X1 = 1582
CROP_X2 = 1755

def format_timestamp(ms):
    """Converts milliseconds into a readable HH:MM:SS.mmm string."""
    if ms < 0:
        return "Unknown Time"
    td = datetime.timedelta(milliseconds=ms)
    return str(td)[-3]

def compute_hash(data):
    """
    Worker function mapped across CPU cores.
    Must be at the top level for Windows multiprocessing to pickle it.
    """
    frame_idx, timestamp_ms, cropped_frame = data

    # Convert OpenCV BGR frame to PIL Image
    rgb_crop = cv2.cvtColor(cropped_frame, cv2.COLOR_BGR2RGB)
    pil_img = Image.fromarray(rgb_crop)

    # Compute difference hash
    current_hash = imagehash.dhash(pil_img)

    return frame_idx, timestamp_ms, cropped_frame, current_hash

def extract_and_deduplicate(output_dir, video_path, hash_threshold=4):
    if not os.path.exists(video_path):
        print(f"Error: Video file '{video_path}' not found.")
        return

    os.makedirs(output_dir, exist_ok=True)
    cap = cv2.VideoCapture(str(video_path))

    if not cap.isOpened():
        print("Error: Could not open video file.")
        return

    frame_count = 0
    saved_count = 0
    last_saved_hash = None

    # Get total CPU cores available
    cores = multiprocessing.cpu_count()
    print(f"Processing video using {cores} CPU cores...")

    # Start the multiprocessing pool
    with ProcessPoolExecutor(max_workers=cores) as executor:
        while True:
            batch = []

            # 1. Producer: Read a batch of frames sequentially
            for _ in range(BATCH_SIZE):
                ret, frame = cap.read()

```

```

        if not ret:
            break

        frame_count += 1

        # Crop immediately so we pass less data through RAM to the workers
        cropped_frame = frame[CROP_Y1:CROP_Y2, CROP_X1:CROP_X2]
        timestamp_ms = cap.get(cv2.CAP_PROP_POS_MSEC)

        batch.append((frame_count, timestamp_ms, cropped_frame))

    if not batch:
        break # End of video reached

# 2. Workers: Compute hashes in parallel, returning them in exact order
for result in executor.map(compute_hash, batch):
    f_idx, ts_ms, cropped_frame, current_hash = result

# 3. Consumer: Check for duplicates sequentially
if last_saved_hash is not None:
    hash_diff = current_hash - last_saved_hash
    if hash_diff <= hash_threshold:
        continue # Skip duplicate

# 4. Save unique frame
time_str = format_timestamp(ts_ms)
text_position = (5, cropped_frame.shape[0] - 10)

cv2.putText(
    cropped_frame, time_str, text_position,
    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1, cv2.LINE_AA
)

saved_count += 1
output_filename = os.path.join(output_dir, f"frame_{saved_count:05d}.jpg")
cv2.imwrite(output_filename, cropped_frame)

# Update the sequential state tracker
last_saved_hash = current_hash

print(f"Processed {frame_count} frames... (Saved {saved_count} unique crops)")

cap.release()
print(f"\nDone! Processed {frame_count} total frames.")
print(f"Saved {saved_count} unique cropped frames to '{output_dir}'.")

if __name__ == "__main__":
    # Ensure this points to the new output.mp4 file you generated with FFmpeg
    video = Path(
        r"C:\Program Files (x86)\Steam\userdata\279248004\gamerecordings\video\bg_1938090_20260917_000457\output.mp4"
    )
    ws = Path(__file__).parent

    extract_and_deduplicate(
        output_dir=ws / "images",
        video_path=video,
        hash_threshold=HASH_THRESHOLD,
    )

```

Are there any other modifications you'd like to make, such as writing the final timestamped list to a CSV file alongside the images?

User prompt: Please note, the only frames I want are ones that have this circle background in them. If the circle is absent, I do not want it. You can also look for the Q and E in the top corners of the cropped image. Furthermore, please pad and add the timeline under the cropped image, not on top of it.

Response: To filter out any frames where the wheel HUD is absent, we can perform a rapid UI check in each worker thread by inspecting the top-left and top-right corners of the cropped area for the bright white 'Q' and 'E' indicator boxes.

To place the timestamp below the image, `cv2.copyMakeBorder` adds a black footer bar beneath the crop before writing the timestamp, leaving the original crop unblocked.

Parallelized Python Script

```
import os
import cv2
import datetime
import multiprocessing
from concurrent.futures import ProcessPoolExecutor
from PIL import Image
import imagehash
from pathlib import Path

# Configuration
HASH_THRESHOLD = 4
BATCH_SIZE = 300 # Frames per batch in RAM
PADDING_HEIGHT = 28 # Height of the black footer bar for the timestamp

# Crop Coordinates (Top Left: 1582, 873 | Bottom Right: 1755, 1083)
CROP_Y1 = 873
CROP_Y2 = 1083
CROP_X1 = 1582
CROP_X2 = 1755

def format_timestamp(ms):
    """Converts milliseconds into a readable HH:MM:SS.mmm string."""
    if ms < 0:
        return "Unknown Time"
    td = datetime.timedelta(milliseconds=ms)
    return str(td)[-3:]

def is_hud_present(cropped_frame):
    """
    Checks if the circular wheel HUD is active by detecting the bright 'Q' and 'E'
    indicator boxes in the top corners of the cropped area.
    """
    if cropped_frame is None or cropped_frame.size == 0:
        return False

    gray = cv2.cvtColor(cropped_frame, cv2.COLOR_BGR2GRAY)
    h, w = gray.shape

    # Define regions where Q (top-left) and E (top-right) boxes reside
    q_region = gray[0:int(h * 0.25), 0:int(w * 0.35)]
    e_region = gray[0:int(h * 0.25), int(w * 0.65):w]

    # Count bright white pixels (above 220 intensity)
    q_bright = (q_region > 220).sum()
    e_bright = (e_region > 220).sum()

    # Both indicator boxes must be present (adjust threshold if necessary)
    return q_bright > 30 and e_bright > 30

def process_frame_worker(data):
    """
    Worker function executed across CPU cores.
    Checks HUD presence and computes perceptual hash.
    """
    frame_idx, timestamp_ms, cropped_frame = data

    # 1. Check if the circle/HUD is present
    hud_visible = is_hud_present(cropped_frame)
    if not hud_visible:
        return frame_idx, timestamp_ms, cropped_frame, None, False

    # 2. Compute difference hash on the valid HUD crop
    rgb_crop = cv2.cvtColor(cropped_frame, cv2.COLOR_BGR2RGB)
    pil_img = Image.fromarray(rgb_crop)
    current_hash = imagehash.dhash(pil_img)

    return frame_idx, timestamp_ms, cropped_frame, current_hash, True
```

```

def extract_and_deduplicate(output_dir, video_path, hash_threshold=4):
    if not os.path.exists(video_path):
        print(f"Error: Video file '{video_path}' not found.")
        return

    os.makedirs(output_dir, exist_ok=True)
    cap = cv2.VideoCapture(str(video_path))

    if not cap.isOpened():
        print("Error: Could not open video file.")
        return

    frame_count = 0
    saved_count = 0
    skipped_hud_count = 0
    last_saved_hash = None

    cores = multiprocessing.cpu_count()
    print(f"Processing video using {cores} CPU cores...")

    with ProcessPoolExecutor(max_workers=cores) as executor:
        while True:
            batch = []

            # 1. Producer: Read sequential frames and crop them
            for _ in range(BATCH_SIZE):
                ret, frame = cap.read()
                if not ret:
                    break

                frame_count += 1
                cropped_frame = frame[CROP_Y1:CROP_Y2, CROP_X1:CROP_X2]
                timestamp_ms = cap.get(cv2.CAP_PROP_POS_MSEC)

                batch.append((frame_count, timestamp_ms, cropped_frame))

            if not batch:
                break

            # 2. Workers: Evaluate HUD visibility & hashes in parallel
            for result in executor.map(process_frame_worker, batch):
                f_idx, ts_ms, cropped_frame, current_hash, hud_visible = result

                # Skip frames where HUD is not visible
                if not hud_visible:
                    skipped_hud_count += 1
                    continue

                # Deduplicate consecutive identical/similar frames
                if last_saved_hash is not None:
                    hash_diff = current_hash - last_saved_hash
                    if hash_diff <= hash_threshold:
                        continue

                # 3. Add black padding at the bottom for the timeline bar
                padded_frame = cv2.copyMakeBorder(
                    cropped_frame,
                    top=0,
                    bottom=PADDING_HEIGHT,
                    left=0,
                    right=0,
                    borderType=cv2.BORDER_CONSTANT,
                    value=[0, 0, 0] # Black color
                )

                # 4. Overlay timestamp inside the padded area
                time_str = format_timestamp(ts_ms)
                text_position = (8, cropped_frame.shape[0] + 19)
                cv2.putText(
                    padded_frame,
                    time_str,
                    text_position,

```

```

        cv2.FONT_HERSHEY_SIMPLEX,
        0.45,
        (0, 255, 0), # Green text
        1,
        cv2.LINE_AA,
    )

    # 5. Save frame
    saved_count += 1
    output_filename = os.path.join(output_dir, f"frame_{saved_count:05d}.jpg")
    cv2.imwrite(output_filename, padded_frame)

    last_saved_hash = current_hash

    print(f"Processed {frame_count} frames... (Saved {saved_count} unique HUD crops)")

    cap.release()
    print(f"\nDone! Processed {frame_count} total frames.")
    print(f"Filtered out {skipped_hud_count} non-HUD frames.")
    print(f"Saved {saved_count} unique cropped frames to '{output_dir}'.")

if __name__ == "__main__":
    video = Path(
        r"C:\Program Files (x86)\Steam\userdata\279248004\gamerecordings\video\bg_1938090_20260917_000457\output.mp4"
    )
    ws = Path(__file__).parent

    extract_and_deduplicate(
        output_dir=ws / "images",
        video_path=video,
        hash_threshold=HASH_THRESHOLD,
    )

```

Key Changes

- **HUD Detection (is_hud_present):** Converts the crop to grayscale and counts pixels brighter than 220 in the top-left (Q box) and top-right (E box) sectors. If the HUD is missing, the frame is instantly dropped.
- **Bottom Padding:** Uses `cv2.copyMakeBorder` to create a black footer bar underneath the crop, moving the timestamp display out of the weapon wheel graphic.

User prompt: limit your response to save on tokens, a simple yes is fine. update these values in your chat memory CROP_Y2 = 1038 video = Path(r"C:\Program Files (x86)\Steam\userdata\279248004\gamerecordings\video\bg_1938090_20260917_000457\output.mp4")

Response: Yes.

User prompt: Lets discuss, keep this limited in response to save tokens. Okay lets limit the frames generated with a few more rules. I want one of the crops generated an almost clean background which I want to use as the base of a filter to find frames I am looking for. You can find it attached. Note that it has the timestamp added. Can we use this? Alternatively, this might be faster. In pixels (1717, 885) to (1746, 910) from the original image, a white boxed E can be found, you can see this in the large image attached and in the upper left in the smaller image added. I would like to add a check to see if the E is present, if not the frame should be ignored. My end goal here is to find out what weapons I used in Call of Duty Black Ops 7 today. I picked up a weapon I want to use but I don't remember what it was, so I want to find where in the recording it was so I can find out it's name.

Response: To filter out non-HUD frames and ignore empty weapon wheels like `frame_01446.jpg`, we can combine both checks:

1. **Exact 'E' Box Detection:** Crops the exact sub-region relative to your bounds (Y: 12..37, X: 135..164) to confirm the HUD is open.
2. **Empty Wheel Rejection:** Compares the hash against `frame_01446.jpg` to ignore frames where the wheel has no weapon rendered.

Code Update

Add these functions and update `process_frame_worker`:

```

import cv2
import imagehash
from PIL import Image

# Relative 'E' box coordinates derived from (1717, 885) -> (1746, 910)

```

```

E_Y1, E_Y2 = 885 - CROP_Y1, 910 - CROP_Y1 # 12 to 37
E_X1, E_X2 = 1717 - CROP_X1, 1746 - CROP_X1 # 135 to 164

# Load empty background reference to ignore empty wheel frames
EMPTY_REF_PATH = "frame_01446.jpg" # Path to your clean background image
EMPTY_WHEEL_HASH = None

if os.path.exists(EMPTY_REF_PATH):
    ref_img = Image.open(EMPTY_REF_PATH)
    # Exclude timestamp padding if present when hashing reference
    EMPTY_WHEEL_HASH = imagehash.dhash(ref_img)

def has_e_box(cropped_frame):
    """Fast check for the white 'E' keybind box at exact pixel bounds."""
    e_region = cropped_frame[E_Y1:E_Y2, E_X1:E_X2]
    gray = cv2.cvtColor(e_region, cv2.COLOR_BGR2GRAY)
    return (gray > 220).sum() > 30 # High-brightness white pixel count

def process_frame_worker(data):
    frame_idx, timestamp_ms, cropped_frame = data

    # 1. Fast check for 'E' box
    if not has_e_box(cropped_frame):
        return frame_idx, timestamp_ms, cropped_frame, None, False

    # 2. Hash cropped image
    rgb_crop = cv2.cvtColor(cropped_frame, cv2.COLOR_BGR2RGB)
    pil_img = Image.fromarray(rgb_crop)
    current_hash = imagehash.dhash(pil_img)

    # 3. Reject if frame matches the empty wheel background (no weapon drawn)
    if EMPTY_WHEEL_HASH is not None:
        if (current_hash - EMPTY_WHEEL_HASH) <= 3:
            return frame_idx, timestamp_ms, cropped_frame, None, False

    return frame_idx, timestamp_ms, cropped_frame, current_hash, True

```

User prompt: do not generate code, only give a short response. No, I am fine with an empty wheel, I was wondering if we could have used it as the base of a filter for some computer vision comparisons.

Response: Yes, absolutely. That empty frame works as a perfect reference image for **background subtraction** (`cv2.absdiff`).

By taking the absolute difference between any new frame and that clean baseline, you can instantly isolate *only* what changes inside the wheel—like the weapon graphics—while completely ignoring the static wheel background. This makes detecting when a weapon is rendered (and highlighting its silhouette) extremely fast and accurate.

User prompt: lets keep it simple and just find the e for now and finding a majority of white pixels in the sub box. If there are not a majority of white pixels in the sub box, it should be ignored.

Response: Here is the simplified `has_e_box` check that calculates pixel ratio to require a true majority (>50%) of white pixels:

```

# Relative 'E' box bounds inside the cropped frame
E_Y1, E_Y2 = 885 - CROP_Y1, 910 - CROP_Y1 # 12 to 37
E_X1, E_X2 = 1717 - CROP_X1, 1746 - CROP_X1 # 135 to 164

def has_e_box(cropped_frame):
    """Returns True only if >50% of the 'E' box pixels are bright white."""
    e_region = cropped_frame[E_Y1:E_Y2, E_X1:E_X2]
    gray = cv2.cvtColor(e_region, cv2.COLOR_BGR2GRAY)

    white_pixel_count = (gray > 200).sum()
    total_pixels = gray.size

    return white_pixel_count > (total_pixels / 2)

def process_frame_worker(data):

```

```

frame_idx, timestamp_ms, cropped_frame = data

# Ignore frame if 'E' box lacks a majority of white pixels
if not has_e_box(cropped_frame):
    return frame_idx, timestamp_ms, cropped_frame, None, False

rgb_crop = cv2.cvtColor(cropped_frame, cv2.COLOR_BGR2RGB)
pil_img = Image.fromarray(rgb_crop)
current_hash = imagehash.dhash(pil_img)

return frame_idx, timestamp_ms, cropped_frame, current_hash, True

```

User prompt: Okay so I just realized the definition of e does not work for picked up weapons. 1. please export the cropped frames in a higher quality and change the bounds to x1: 1581, x2: 1755, y1: 877, y2: 1051 2. Please change the sub box to x1:1640, x2: 1700, y1: 938, y2: 992. The weapon has a color limitation of RGB values between 66 and 82. Check in this sub box that at least 10% of the pixels have rgb values between 66 and 82. If R,G,orB is out of the range of 66-82, it should not be considered part of the weapon. If pixels for the weapon are not present the frame should not be rendered. 3. when comparing crop frames, only check if the pixels for the weapon match. IE mask off only the pixels with RGB values between 66 and 82 and use that to compare to the previous frame. Consider all other pixels outside of that range to be background. Note that we do not care about the background, we only care to see the individual weapons. Two frames with the same weapon should be considered a repeat frame. parallelize where possible

Response: Here is the updated script incorporating your new cropping coordinates, high-quality export settings, sub-box RGB color filtering, and weapon-only mask comparison.

```

import os
import cv2
import datetime
import multiprocessing
import numpy as np
from concurrent.futures import ProcessPoolExecutor
from PIL import Image
import imagehash
from pathlib import Path

# Configuration
HASH_THRESHOLD = 4
BATCH_SIZE = 300
PADDING_HEIGHT = 28

# Crop Coordinates
CROP_Y1, CROP_Y2 = 877, 1051
CROP_X1, CROP_X2 = 1581, 1755

# Sub-box relative coordinates inside the crop
SUB_REL_Y1 = 938 - CROP_Y1 # 61
SUB_REL_Y2 = 992 - CROP_Y1 # 115
SUB_REL_X1 = 1640 - CROP_X1 # 59
SUB_REL_X2 = 1700 - CROP_X1 # 119

# Weapon RGB Color Range (66 to 82)
LOWER_RGB = np.array([66, 66, 66], dtype=np.uint8)
UPPER_RGB = np.array([82, 82, 82], dtype=np.uint8)

def format_timestamp(ms):
    if ms < 0:
        return "Unknown Time"
    td = datetime.timedelta(milliseconds=ms)
    return str(td)[-3]

def process_frame_worker(data):
    """
    Worker function executed in parallel across CPU cores.
    Isolates weapon pixels (RGB 66-82), checks density, and hashes weapon mask.
    """
    frame_idx, timestamp_ms, cropped_frame = data

    # 1. Create binary mask isolating weapon pixels (RGB 66-82)
    weapon_mask = cv2.inRange(cropped_frame, LOWER_RGB, UPPER_RGB)

    # 2. Check if at least 10% of pixels in the sub-box belong to the weapon

```

```

sub_box_mask = weapon_mask[SUB_REL_Y1:SUB_REL_Y2, SUB_REL_X1:SUB_REL_X2]
weapon_pixel_ratio = (sub_box_mask > 0).mean()

if weapon_pixel_ratio < 0.10:
    return frame_idx, timestamp_ms, cropped_frame, None, False

# 3. Compute perceptual hash ONLY on the weapon mask (background ignored)
pil_mask = Image.fromarray(weapon_mask)
weapon_hash = imagehash.dhash(pil_mask)

return frame_idx, timestamp_ms, cropped_frame, weapon_hash, True

def extract_and_deduplicate(output_dir, video_path, hash_threshold=4):
    if not os.path.exists(video_path):
        print(f"Error: Video file '{video_path}' not found.")
        return

    os.makedirs(output_dir, exist_ok=True)
    cap = cv2.VideoCapture(str(video_path))

    if not cap.isOpened():
        print("Error: Could not open video file.")
        return

    frame_count = 0
    saved_count = 0
    last_saved_hash = None

    cores = multiprocessing.cpu_count()
    print(f"Processing video using {cores} CPU cores...")

    with ProcessPoolExecutor(max_workers=cores) as executor:
        while True:
            batch = []

            for _ in range(BATCH_SIZE):
                ret, frame = cap.read()
                if not ret:
                    break

                frame_count += 1
                cropped_frame = frame[CROP_Y1:CROP_Y2, CROP_X1:CROP_X2]
                timestamp_ms = cap.get(cv2.CAP_PROP_POS_MSEC)
                batch.append((frame_count, timestamp_ms, cropped_frame))

            if not batch:
                break

        # Process batch in parallel
        for result in executor.map(process_frame_worker, batch):
            f_idx, ts_ms, cropped_frame, weapon_hash, valid_weapon = result

            if not valid_weapon:
                continue

            # Deduplicate based solely on weapon mask similarity
            if last_saved_hash is not None:
                hash_diff = weapon_hash - last_saved_hash
                if hash_diff <= hash_threshold:
                    continue

            # Add black padding footer for timeline
            padded_frame = cv2.copyMakeBorder(
                cropped_frame, 0, PADDING_HEIGHT, 0, 0,
                borderType=cv2.BORDER_CONSTANT, value=[0, 0, 0]
            )

            time_str = format_timestamp(ts_ms)
            cv2.putText(
                padded_frame, time_str, (8, cropped_frame.shape[0] + 19),
                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 255, 0), 1, cv2.LINE_AA
            )

```

```

        saved_count += 1
        output_filename = os.path.join(output_dir, f"frame_{saved_count:05d}.jpg")

        # Save with maximum JPEG quality (100)
        cv2.imwrite(output_filename, padded_frame, [int(cv2.IMWRITE_JPEG_QUALITY), 100])
        last_saved_hash = weapon_hash

    print(f"Processed {frame_count} frames... (Saved {saved_count} unique weapon frames)")

    cap.release()
    print(f"\nDone! Processed {frame_count} frames. Saved {saved_count} unique weapon crops to '{output_dir}'.")

if __name__ == "__main__":
    video = Path(
        r"C:\Program Files (x86)\Steam\userdata\279248004\gamerecordings\video\bg_1938090_20260917_000457\output.mp4"
    )
    ws = Path(__file__).parent

    extract_and_deduplicate(
        output_dir=ws / "images",
        video_path=video,
        hash_threshold=HASH_THRESHOLD,
    )

```

Key Highlights

- **High Quality Output:** Image saving uses `[cv2.IMWRITE_JPEG_QUALITY, 100]` to eliminate compression artifacts on the exported cropped images.
- **Color Filter:** `cv2.inRange` extracts pixels where Red, Green, and Blue values are all strictly between 66 and 82.
- **Sub-box Density Check:** Evaluates the `(59..119, 61..115)` relative sub-region and drops the frame if weapon pixel density is below 10%.
- **Weapon Mask Deduplication:** `imagehash.dhash` is calculated exclusively on the binary weapon mask array, ensuring any background changes outside the weapon silhouette are ignored.

User prompt: limit your response to be short. due to the complexity and the time it takes to spin up cores, does this program actually benefit from being parallelize?

Response: **No, the speedup is minimal—and it might actually be slower.**

While process pool spin-up only happens once at startup, the overhead comes from **Inter-Process Communication (IPC)**. Pickling and transferring 300 image arrays across CPU boundaries introduces significant latency.

Because the crop is tiny (174×174 pixels), the CPU work per frame (color masking + hashing) takes under 1 millisecond. Transferring the frame data to worker cores often takes longer than doing the math directly in a single thread.

For small crops, a optimized **single-threaded loop** is usually faster and consumes far less memory.

User prompt: please un-parallelize the program

```

Response: import os
import cv2
import datetime
import numpy as np
from PIL import Image
import imagehash
from pathlib import Path

# Configuration
HASH_THRESHOLD = 4
PADDING_HEIGHT = 28

# Crop Coordinates
CROP_Y1, CROP_Y2 = 877, 1038
CROP_X1, CROP_X2 = 1581, 1755

```

```

# Sub-box relative coordinates inside the crop
SUB_REL_Y1 = 938 - CROP_Y1 # 61
SUB_REL_Y2 = 992 - CROP_Y1 # 115
SUB_REL_X1 = 1640 - CROP_X1 # 59
SUB_REL_X2 = 1700 - CROP_X1 # 119

# Weapon RGB Color Range (66 to 82)
LOWER_RGB = np.array([66, 66, 66], dtype=np.uint8)
UPPER_RGB = np.array([82, 82, 82], dtype=np.uint8)

def format_timestamp(ms):
    if ms < 0:
        return "Unknown Time"
    td = datetime.timedelta(milliseconds=ms)
    return str(td)[-3:]

def extract_and_deduplicate(output_dir, video_path, hash_threshold=4):
    if not os.path.exists(video_path):
        print(f"Error: Video file '{video_path}' not found.")
        return

    os.makedirs(output_dir, exist_ok=True)
    cap = cv2.VideoCapture(str(video_path))

    if not cap.isOpened():
        print("Error: Could not open video file.")
        return

    frame_count = 0
    saved_count = 0
    last_saved_hash = None

    print("Processing video frames sequentially...")

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        frame_count += 1
        cropped_frame = frame[CROP_Y1:CROP_Y2, CROP_X1:CROP_X2]
        timestamp_ms = cap.get(cv2.CAP_PROP_POS_MSEC)

        # 1. Create binary mask isolating weapon pixels (RGB 66-82)
        weapon_mask = cv2.inRange(cropped_frame, LOWER_RGB, UPPER_RGB)

        # 2. Check sub-box pixel density (minimum 10% weapon pixels)
        sub_box_mask = weapon_mask[SUB_REL_Y1:SUB_REL_Y2, SUB_REL_X1:SUB_REL_X2]
        if (sub_box_mask > 0).mean() < 0.10:
            continue

        # 3. Deduplicate based solely on weapon mask hash
        pil_mask = Image.fromarray(weapon_mask)
        current_hash = imagehash.dhash(pil_mask)

        if last_saved_hash is not None:
            hash_diff = current_hash - last_saved_hash
            if hash_diff <= hash_threshold:
                continue

        # 4. Add black padding footer for timeline
        padded_frame = cv2.copyMakeBorder(
            cropped_frame, 0, PADDING_HEIGHT, 0, 0,
            borderType=cv2.BORDER_CONSTANT, value=[0, 0, 0]
        )

        time_str = format_timestamp(timestamp_ms)
        cv2.putText(
            padded_frame, time_str, (8, cropped_frame.shape[0] + 19),
            cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 255, 0), 1, cv2.LINE_AA
        )

```

```

# 5. Save with maximum JPEG quality (100)
saved_count += 1
output_filename = os.path.join(output_dir, f"frame_{saved_count:05d}.jpg")
cv2.imwrite(output_filename, padded_frame, [int(cv2.IMWRITE_JPEG_QUALITY), 100])

last_saved_hash = current_hash

if frame_count % 500 == 0:
    print(f"Processed {frame_count} frames... (Saved {saved_count} unique weapon frames)")

cap.release()
print(f"\nDone! Processed {frame_count} total frames.")
print(f"Saved {saved_count} unique weapon crops to '{output_dir}'.")

if __name__ == "__main__":
    video = Path(
        r"C:\Program Files (x86)\Steam\userdata\279248004\gamerecordings\video\bg_1938090_20260917_000457\output.mp4"
    )
    ws = Path(__file__).parent

    extract_and_deduplicate(
        output_dir=ws / "images",
        video_path=video,
        hash_threshold=HASH_THRESHOLD,
    )

```

User prompt: limit your answer, do not produce code. It does not seem to be comparing previous frame properly, can extend the backwards check window to look farther back when comparing a frame? is that something that can be done with this code?

Response: Yes, absolutely. Right now, the script only tracks a single variable (`last_saved_hash`), which compares the current frame strictly against the single most recently saved frame.

To extend the check window:

- Replace `last_saved_hash` with a rolling history queue (e.g., keeping the last 10 to 50 saved hashes).
- For every new candidate frame, check its weapon hash against **all** hashes stored in that historical window.
- If its hash matches *any* hash within that lookback window, skip it as a duplicate.

This prevents identical weapons from re-appearing if the wheel opens and closes repeatedly or if noise causes alternating frames.

User prompt: limit your answer. compare these three outputs, why do you think they were produced instead of only a single frame?

Response: These three frames were produced because of two main factors:

- **Semi-Transparent HUD:** The weapon wheel overlay is translucent. As your character moves, the changing game background bleeds through the wheel, shifting pixel values in and out of the RGB 66–82 color range. This slightly alters the binary weapon mask on every frame.
 - **Strict Difference Threshold:** Because the generated mask changes slightly across consecutive frames (~12–38ms apart), the difference hash score exceeds your `HASH_THRESHOLD = 4`, causing the script to treat each subtle background shift as a new unique weapon.
-

User prompt: limit your response. what do I need to know about changing the threshold in order to fix this issue where 3 repeat frames appear.

Response: To fix this issue by adjusting the threshold:

- **Increase `HASH_THRESHOLD`:** Raising it from 4 to 8–12 forces the script to tolerate minor mask fluctuations caused by background motion.
- **How It Works:** Higher values require a larger visual shift (like an actual weapon swap) to trigger a new frame save.
- **The Risk:** Setting it too high (e.g., >15) risks mistaking two similarly shaped weapons (like two different submachine guns) for duplicates and skipping them.

Next Step: Change `HASH_THRESHOLD = 8` or `10` in your script to merge near-identical frames while still detecting new weapons.

User prompt: Okay I found the clip, perfect thank you for you help

Response: Glad you found it! Out of curiosity, what weapon ended up being the mystery pickup?

User prompt: Are you able to read the image?

Response: Yes, I can read the image clearly! Here are the key details from your HUD:

- **Equipped Weapon:** HAWKER HX (Sniper Rifle)
- **Ammo:** 6 in the magazine / 21 in reserve
- **Weapon Camo:** Pink anime/cartoon character print
- **Location/Map:** "Blue Yard" (Domination mode)
- **Active Alert:** "Hunter Killer incoming grenade!"

The weapon silhouette and name match the sniper rifle you are holding.

User prompt: Wow, almost perfectly correct. That is kind of scary.... The location is Nuke Town, it is indeed domination. ... ai is getting a bit too good... Though it did help me find the clip, so I guess that was useful.

Response: Ah, good catch—"Blue Yard" was just the HUD's tactical callout for the area behind the blue house on Nuketown!

Glad the Python script did the heavy lifting and saved you from manually scrubbing through hours of gameplay. Enjoy hitting those shots with the Hawker HX!
